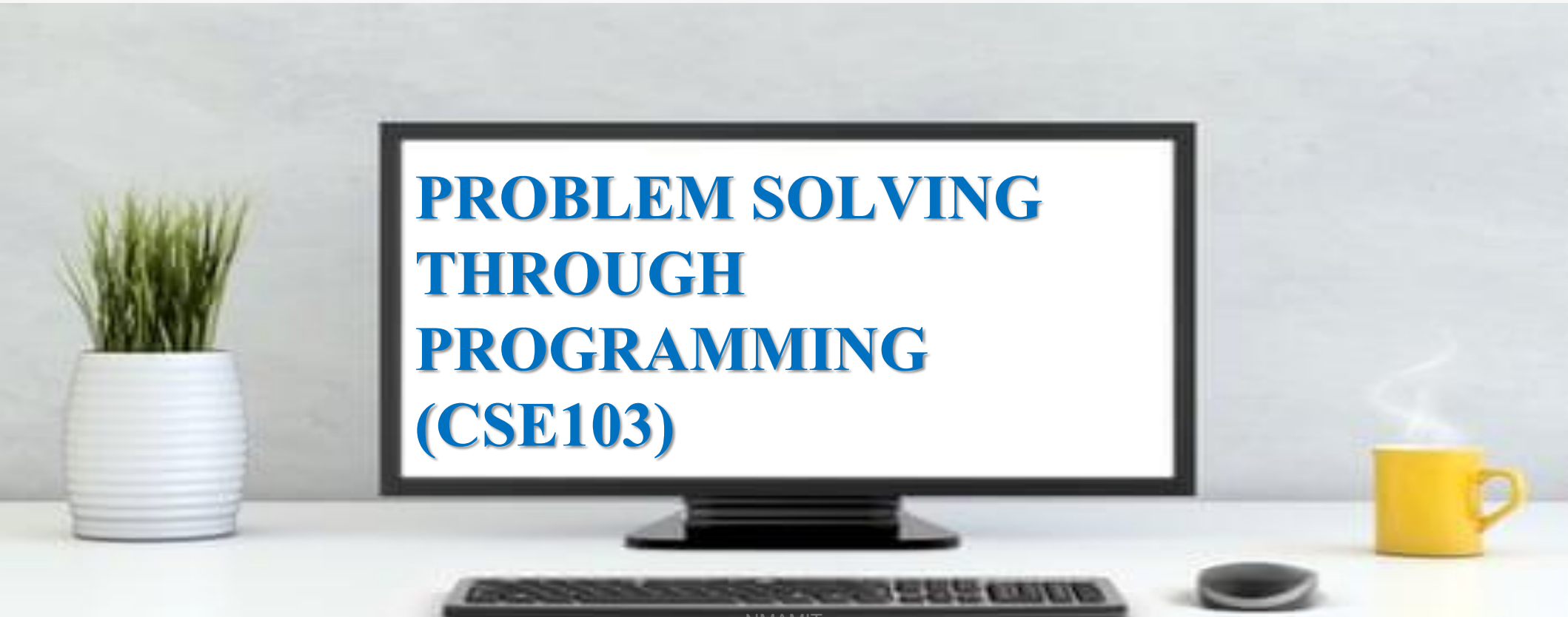




NITTE

(Deemed to be University)

NMAM INSTITUTE OF TECHNOLOGY



PROBLEM SOLVING THROUGH PROGRAMMING (CSE103)

Course Outcomes

Describe the basics of computer systems, basics of C and the process of problem- solving aspects using algorithmic solutions for a given problem.

Apply the knowledge to develop the C program using control statements such as branching and looping constructs for a given problem.

Demonstrate the ability to implement and analyze operations on arrays and strings to solve computational problems.

Apply the concept of pointers along with code reusability, parameter passing, and return values to develop efficient and maintainable C programs using functions.

Implement and demonstrate the use of structures and file handling mechanisms in C programs.

UNIT 1:

Introduction to Computer and C Language, Operators and Expressions

Unit 1 Syllabus:

Introduction to computers:

Introduction to Computer System and problem solving, Program Development steps, Introduction to Programming Languages, input and output devices, Algorithm, Flowchart, Designing efficient programs. Evolution & Characteristics of C Language, Structure of a C Program, C Compilation Model. Characters set, C tokens, Keywords and identifiers, Constants, Variables, Data Types, and Declaration of Variables. Formatted and Unformatted Input/Output

:Formatted I/O: printf() ,scanf() ,Unformatted I/O: getchar(), putchar() ,gets() and puts(), Command line arguments.

Operators and Expressions: Arithmetic operators, Relational operators, Logical operators, Assignment operators, Increment, Decrement operators, Conditional(Ternary) operator, Bitwise operators, Special Operators. Arithmetic expressions, Evaluation of expressions, Precedence of operators, Type conversions in expressions, Operator precedence and associativity.

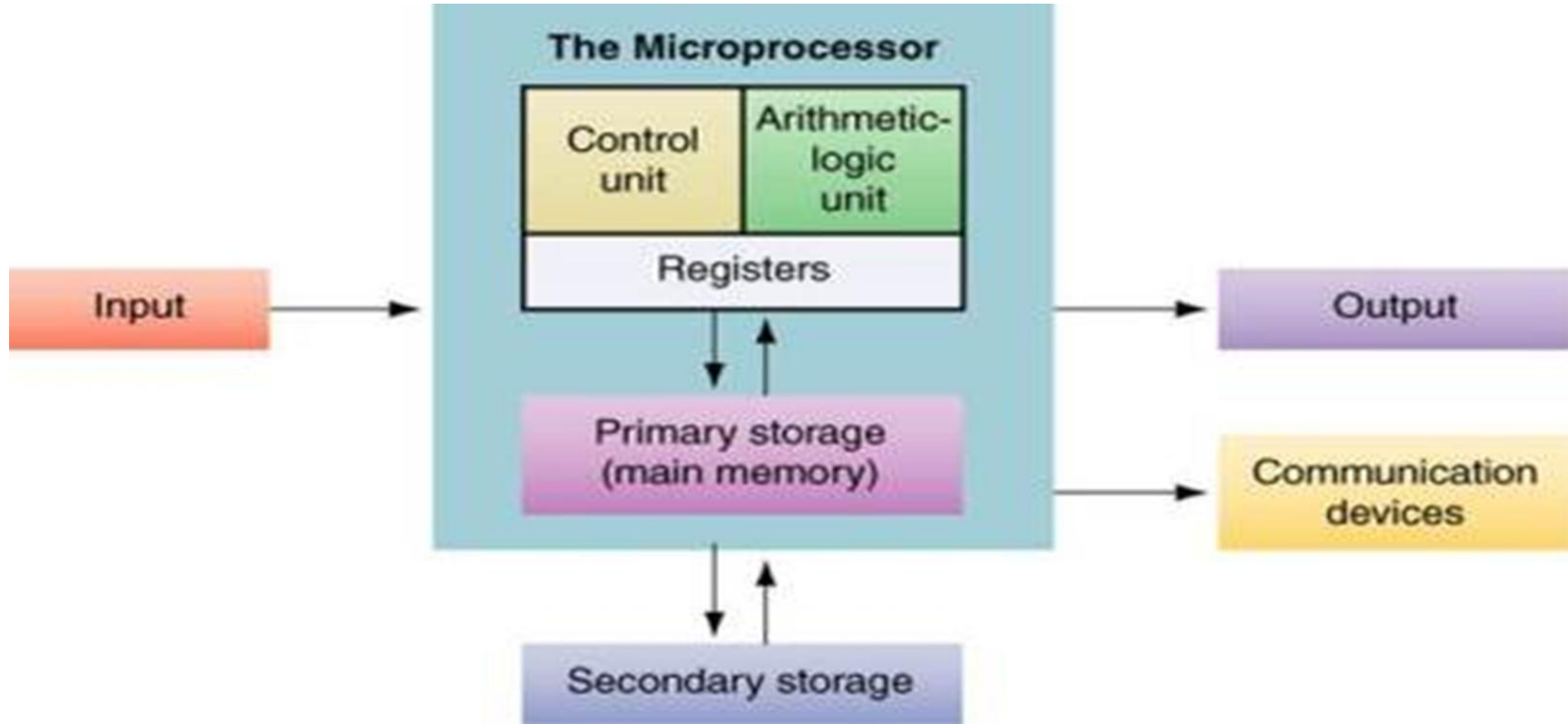
Introduction to Computer Systems & Problem Solving

Computer Systems

- Computer is an electronic device which reads data from input devices and process the data based on the user instructions and displays the output-on-output devices.
- A computer system is a complete, functional setup that includes hardware, software, data, and users working together to perform tasks, process information, and produce results.
- Components of a Computer System:
 - Hardware: Physical components that make up the computer system.
 - Software: Programs and data that tell the hardware what to do.
 - Users: People who operate and interact with the computer system.



Components of a Computer System



Components of a Computer System

Input

- Input refers to the data and instructions that are entered into the computer system using input devices such as a keyboard, mouse, scanner, or microphone.
- These devices allow users to communicate with the computer.
- Once input is received, it is passed to the central part of the system for processing.
- Without input devices, the system would not be able to receive external information or user commands.

Input Devices



Keyboard



Mouse



Microphone



Webcam



Touch Screen



Scanner

Components of a Computer System

The Microprocessor (CPU):

- At the heart of the computer system lies the microprocessor, also known as the Central Processing Unit (CPU).
- It is the main processing unit responsible for executing instructions and managing the flow of data.
- The microprocessor performs all calculations and decision-making activities.
- It consists of three main parts: the **Control Unit**, the **Arithmetic Logic Unit**, and **Registers**.

Components of a Computer System

The Microprocessor (CPU):

- If a CPU receives input that says to complete a calculation or logical operation, this is done with the **arithmetic logic unit(ALU)**.
- If the instructions are to retrieve something from memory, it will use the **control unit** to find the data and translate it from binary into the necessary format for the requested output.
- If the input is simply to store data, then this information is moved into the **memory unit**.
- **Registers** are small, high-speed storage units located within the CPU. They temporarily store data and instructions that are currently being used or processed.

Components of a Computer System

Primary Storage (Main Memory):

- Primary memory is the computer memory that is directly accessible by CPU.
- It holds the data and instructions that the processor is currently working on. As primary memory is an internal memory of the computer, accessing data from this memory is faster.
- This memory is the most volatile because the data in primary memory does not reside if data is not saved when a power cut occurs.
- Primary memory is costlier as compared with secondary memory.
- Its capacity is very much limited and is smaller than the secondary memory.

Components of a Computer System

Secondary Memory / Mass Storage:

- The contents of the secondary memory first get transferred to the primary memory and then are accessed by the processor, this is because the processor does not directly interact with the secondary memory.
- All devices/computers which are capable of storing high volume data are referred to as secondary memory.
- These types of memories are slower than primary memory.
- However, it can save a large amount of data, in the range of Gigabytes (GB) to Terabytes (TB).
- Backup storage or mass storage media are the other names for Secondary memory.

Components of a Computer System

Output devices:

- Output devices are used to display or present the results of processing to the user.
- Common examples include monitors, printers, and speakers.
- Once the CPU has processed the input data, the output devices allow users to see or hear the results in a human-readable or usable form.

Communication Devices:

- Communication devices allow the computer system to send and receive data from other systems or networks.
- These include devices like modems, network interface cards (NICs), and wireless adapters.

Output Devices



MONITOR



PRINTER



SPEAKER

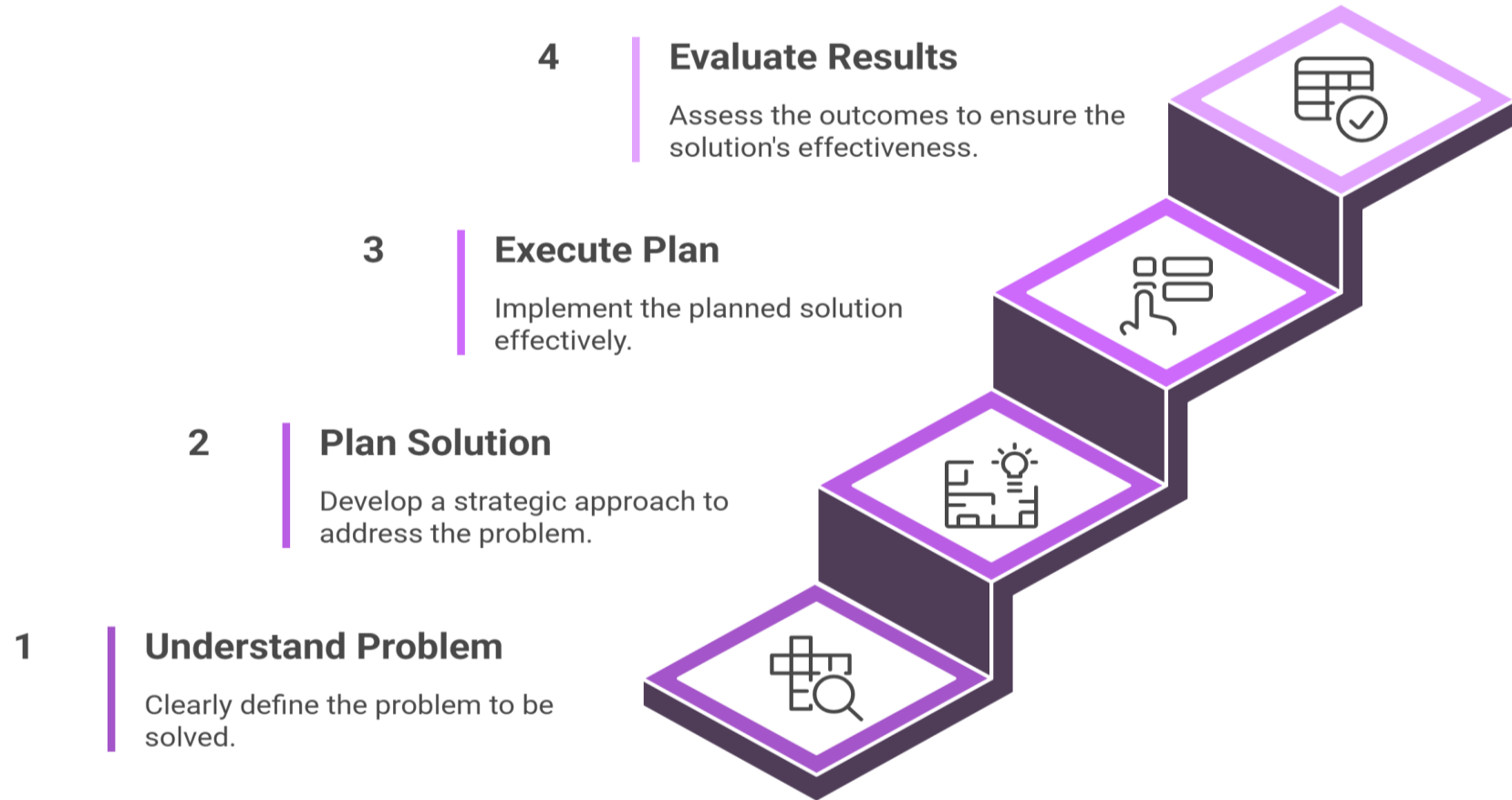


HEADPHONE

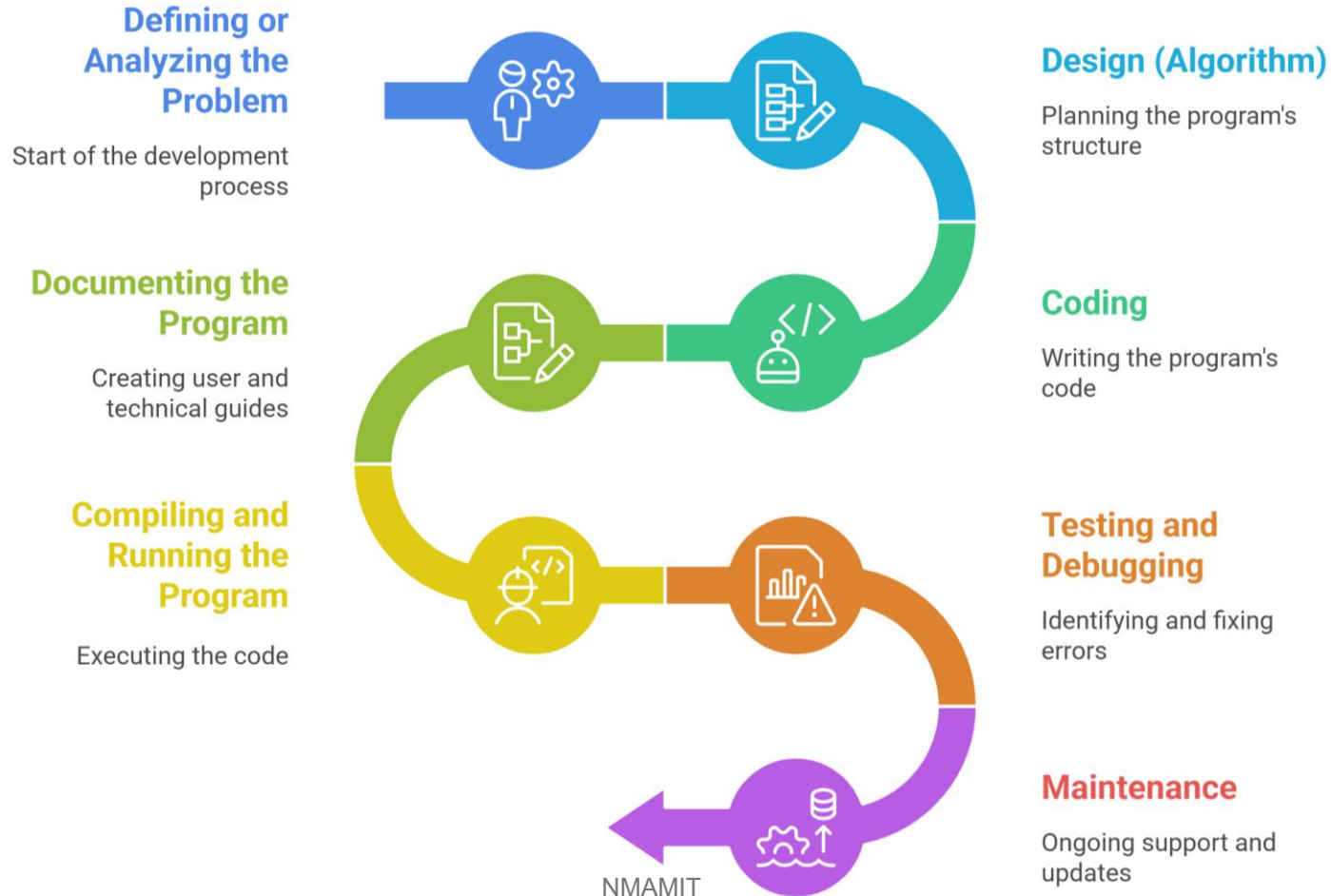


PROJECTOR

Steps to Effective Problem Solving



Program Development Steps



Program Development Steps

Defining or Analyzing the Problem:

- This is the most crucial step. It involves understanding what the problem is, identifying inputs, desired outputs, and the processing required. The goal is to gather complete and clear requirements so the developer knows exactly what needs to be done.

Design (Algorithm):

- Once the problem is understood, the next step is to design a logical plan to solve it.
- This includes:
 - Writing an algorithm (step-by-step procedure)
 - Creating flowcharts or pseudocode to visualize the logic.

Program Development Steps

Coding:

- In this phase, the designed algorithm is translated into a programming language (like C, Python, or Java). The developer writes the actual code using the selected language's syntax and rules.

Documenting the Program:

- Good programming practice involves writing clear documentation. This means:
 - Adding comments in the code
 - Writing user manuals or guides
 - Explaining the logic and structure

Program Development Steps

Compiling and Running the Program:

- After coding, the program is compiled to check for syntax errors.
- If there are no errors, the program is executed to see if it performs the desired task. This phase helps convert the human-written code into machine-understandable form.

Testing and Debugging:

- The program is now tested with different sets of data to ensure it works correctly in all scenarios. If bugs (errors) are found, they are debugged identified and fixed to improve program correctness and stability.

Maintenance:

- After deployment, the program may need updates, error fixes, or feature enhancements.
- Maintenance ensures the software continues to meet user needs over time, even as requirements change or new issues arise.

Introduction to Programming Languages

What is a Programming Language?

- Computer Programming is an art of making a computer to do the required operations, by means of issuing sequence of commands to it.
- Ex: C, C++, JavaScript, C#,HTML.

Types of Programming Languages

- There are two major types of programming languages:



Low-Level Languages

Assembly, Machine Code

Closer to hardware, more difficult to write



High-Level Languages

Python, Java, C++

Easier to read and write, closer to human language

Introduction to Programming Languages

Low Level Languages

- These are machine oriented and require extensive knowledge of computer hardware architecture and its configuration.
- Low Level languages are further divided in to Machine language and Assembly language.

Machine Language

- Only language that is directly understood by the computer. It does not need any translator program.
- The instructions are called machine instruction (machine code) and it is written as strings of 1's (one) and 0's (zero). When this sequence of codes is fed in to the computer, it recognizes the code and converts it in to electrical signals.
- Example : A program instruction in machine language may look like this: 1011000111101

Advantage

- The program runs faster because no translation is needed.

Disadvantages

- It is very difficult to write programs and debug the program.

Introduction to Programming Languages

Assembly Language

- In assembly language, set of mnemonics (symbolic keywords) are used to represent machine codes. Mnemonics are usually combination of words like ADD, SUB and LOAD etc..
- In order to execute the programs written in assembly language, a translator program is required to translate it to the machine language. This translator program is called Assembler.

Advantages

- The symbolic keywords are easier to code and saves time and effort
- It is easier to correct errors and modify programming instructions
- Assembly Language has utmost the same efficiency of execution as the machine level language, because there is one- to-one translation between assembly language program and its corresponding machine language program

Disadvantages

Assembly languages are machine dependent.

A program written for one computer might not run in another computer.

Introduction to Programming Languages

High Level Languages

- High level languages are the simple languages that use English like instructions and mathematical symbols like +, -, %, /, for its program construction.
- In high level languages, it is enough to know the logic and required instructions for a given problem, irrespective of the type of computer used.
- Compiler is a translator program which converts a program in high level language in to machine language.
- Higher level languages are problem-oriented languages because the instructions are suitable for solving a particular problem.
- Examples : COBOL, BASIC, FORTRAN, C, C++, etc..

Advantages

- High level languages are easy to learn and use

Introduction to Programming Languages

Why Study Programming Languages?

- The design of new programming languages and implementation methods have been evolved and improved to meet the change in requirements. Thus, there are many new languages.
- The study of more than one programming language helps us to:
 - master different programming paradigms
 - enhance the skills to state different programming concepts
 - understand the significance of a particular language implementation
 - compare different languages and to choose appropriate language
 - improve the ability to learn new languages and to design new languages

Algorithms

An algorithm is a step-by-step procedure to solve a given problem. It is a finite set of unambiguous instructions that, when executed, perform a task correctly. There are 3 important features,

- The number of steps required to perform the task should be finite
- Each of the instructions in the algorithm should be unambiguous in nature
- Finally, the algorithm should solve the problem correctly.

Statements in C programming language

1. Sequential or imperative statements (actions)
2. Conditional / Selection statements (decision making)
3. Looping / Iterative statements

Develop an algorithm to find the average of 3 numbers taken as input from user (use of imperative statements)

Step 0 START

Step 1 INPUT first number into variable A

Step 2 INPUT second number into variable B

Step 3 INPUT third number into variable c

Step 4 COMPUTE $SUM = A + B + C$

Step 5 COMPUTE $AVG = SUM / 3$

Step 6 DISPLAY AVG

Step 7 END

Develop an algorithm to find maximum of 2 numbers input by the user (use of Conditional statements)

Step 0 START

Step 1 INPUT first number into variable A

Step 2 INPUT second number into variable B

Step 3 IF $A > B$ THEN

 MAX = A

Else

 MAX = B

END IF

Step 4 DISPLAY MAX

Step 5 END

Develop an algorithm to divide one number by another and find the quotient (use of Conditional statements)

- Step 0 START
- Step 1 INPUT first number into a variable A
- Step 2 INPUT second number into variable B
- Step 3 IF $B \neq 0$ THEN

$Q = A/B$

DISPLAY Q

END IF

Step 4 END

Develop an algorithm to find the sum of the first N natural numbers, where N would be input by the user. (Using Looping / Iteration)

Step 0 Start

Step 1 Input N

Step 2 $I=1$

Step 3 $SUM=0$

Step 4 REPEAT Steps (a) to (b) WHILE $I \leq N$

(a) $SUM = SUM + 1$

(b) $I = I + 1$

END WHILE

STEP 5 DISPLAY SUM

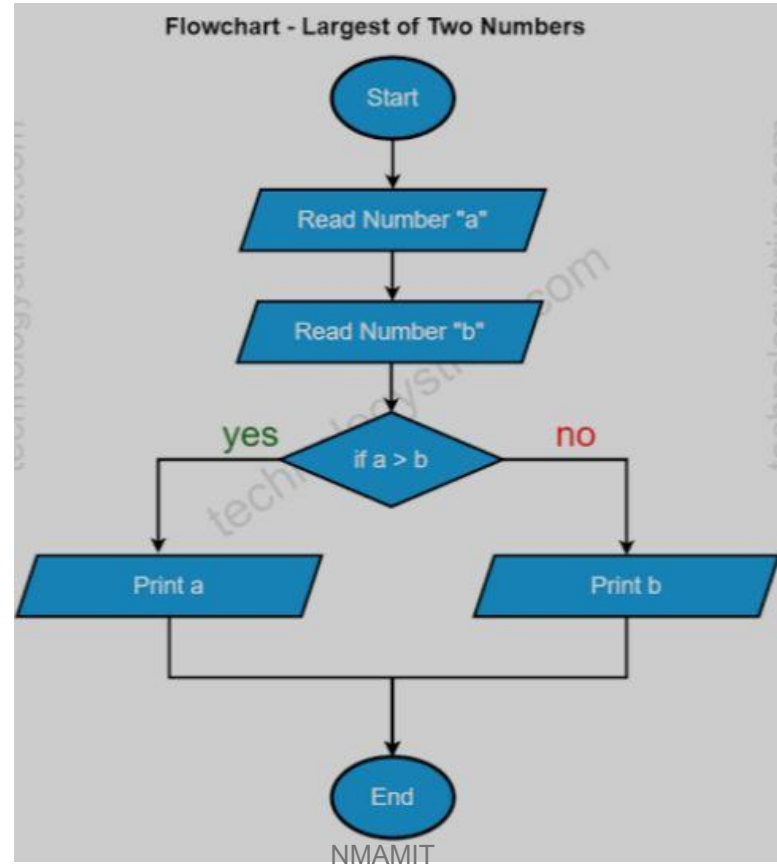
STEP 6 END

FLOWCHARTS

- Flowchart is a graphical representation of an algorithm. Programmers often use it as a program-planning tool to solve a problem.
- It makes use of symbols which are connected among them to indicate the flow of information and processing.

Symbol	Function	Description
	Start / End	An oval represents a start or end of code
	Flow Arrow	Arrows are used to represent the code flow between the flow shapes
	Input / Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

Sample Flowchart



Rules For Creating a Flowchart

A flowchart is a graphical representation of an algorithm. It should follow some rules while creating a flowchart

Rule 1: Flowchart opening statement must be 'start' keyword.

Rule 2: Flowchart ending statement must be 'end' keyword.

Rule 3: All symbols in the flowchart must be connected with an arrow line.

Rule 4: Each decision point should have two or more distinct outcomes.

Rule 5: Flow should generally move from top to bottom or left to right.

Develop an algorithm to find the average of 3 numbers taken as input from user

Algorithm

Step 0 START

Step 1 INPUT first number into variable A

Step 2 INPUT second number into variable B

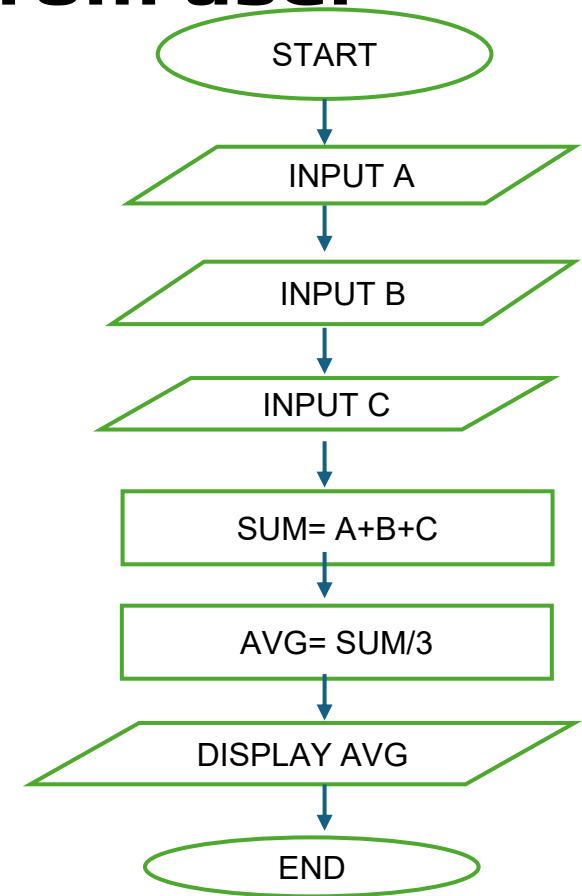
Step 3 INPUT third number into variable c

Step 4 COMPUTE $SUM = A + B + C$

Step 5 COMPUTE $AVG = SUM / 3$

Step 6 DISPLAY AVG

Step 7 END



Develop an algorithm to divide one number by another and find the quotient

Algorithm

Step 0 START

Step 1 INPUT first number into variable A

Step 2 INPUT second number into variable B

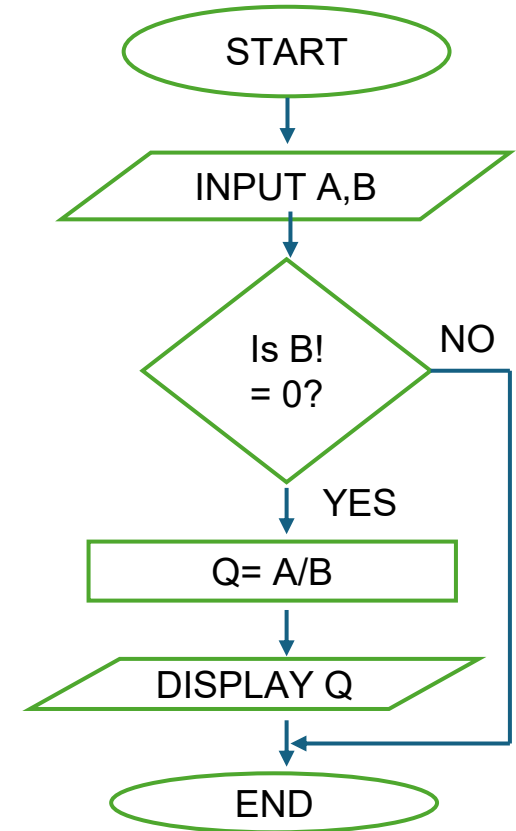
Step 3 If $B \neq 0$ THEN

$Q = A/B$

DISPLAY Q

END IF

Step 4 END



Develop an algorithm to find the maximum of two numbers input by the user

Algorithm

Step 0 START

Step 1 INPUT first number into variable A

Step 2 INPUT second number into variable B

Step 3 If $A > B$ THEN

$MAX = A$

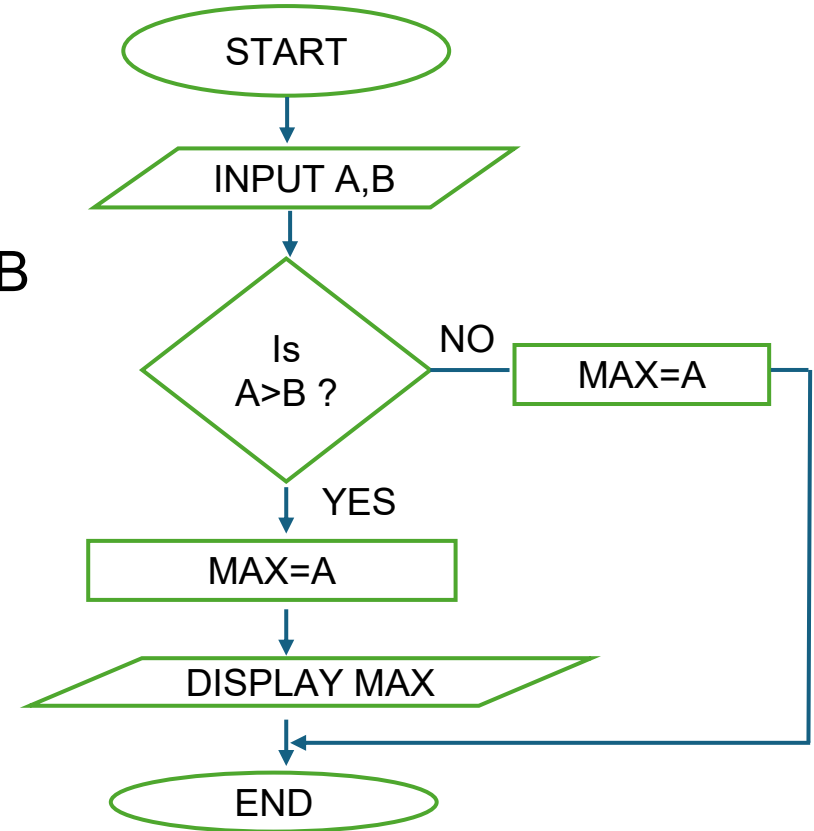
ELSE

$MAX = B$

END IF

Step 4 DISPLAY MAX

Step 5 END



Designing efficient programs

1. Plan before you code
2. Choose the right tools
3. Following coding standards
4. Write modular and reusable codes
5. Test and debug your code
6. Learn from others

Reference

<https://www.linkedin.com/advice/1/how-do-you-program-more-efficiently-skills-information-technology>

Evolution & Characteristics of C Language

- C was developed at Bell Labs by Dennis Ritchie for the Unix Operating System (OS) in the early 1970s.
- C provides a variety of data types.
- In 1972 Dennis Ritchie at Bell Labs writes C and in 1978 the publication of the C Programming Language by Kernighan & Ritchie caused a revolution in the computing world. Origin of C
- Evolved from two previous languages – BCPL, B
- BCPL (Basic Combined programming language) used for writing OS & compilers B used for creating early versions of UNIX OS
- Both are “typeless” language
- C language evolved from B (Dennis Ritchie – Bell labs)

- C is a high-level and general purpose programming language that is ideal for developing firmware or portable applications.
- C is what is called a compiled language.
- Today C is the most widely used System Programming Language.
- Most of the state of the art software have been implemented using C

Why C?

- C was initially used for system development work, in particular the programs that make-up the operating system.
- C was adopted as a system development language because it produces code that runs nearly as fast as code written in assembly language.
- Some examples of the use of C might be: • Operating Systems • Language Compilers • Network Drivers • Modern Programs • Data Bases • Utilities

- C is a structured programming language C supports functions that enables easy maintainability of code, by breaking large file into smaller modules.
- Comments in C provides easy readability.
- C is a powerful language.
- One can say, C is a base for the programming.
- If you know 'C,' you can easily grasp the knowledge of the other programming languages that uses the concept of 'C'

Characteristics of C Language

- 'C' is a **general purpose** programming language.
- 'C' is a **structured** programming language.
- 'C' is **Top Down** Approach
- 'C' is highly **portable**.
- 'C' has rich set of built-in-functions and operators
- 'C' supports various **data types**
- 'C' has **keywords**
- 'C' is **case sensitive**, for example num is different from NUM.
- 'C' supports **pointers** and operations on pointers.

C is considered a **procedure-oriented programming language** (POP). This means it focuses on procedures or routines (functions) to operate on data, organizing code around functions and procedures. Key characteristics of procedure-oriented languages include:

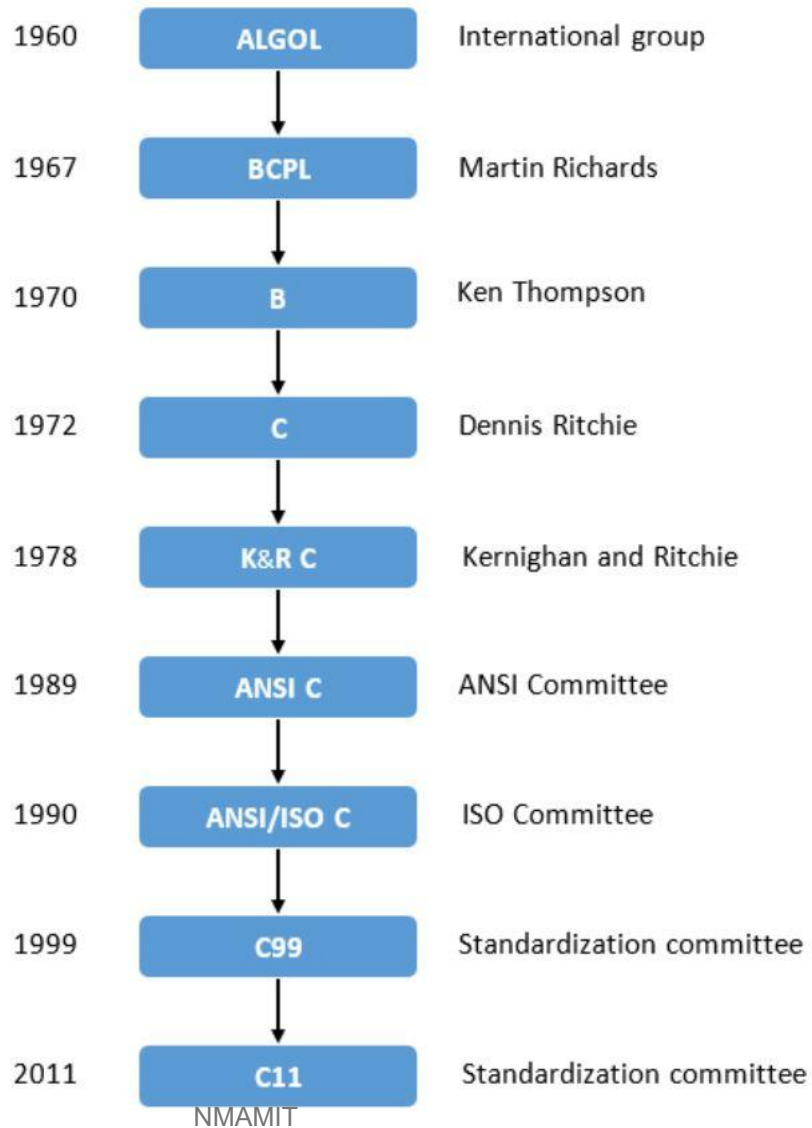
1.Modular Programming: Code is divided into small modules or functions, which can be reused.

2. Top-down Approach: The design starts from the top, specifying complex operations first, and breaking them down into smaller operations.

3.Global Data: Data can be shared among functions.

4. Control Structures: Use of loops, conditionals, and other control structures to dictate the flow of the program.

History of ANSI C



Basic structure of C program:

Structure of C program is defined by set of rules called protocol, to be followed by programmer while writing C program. All C programs are having sections/parts which are mentioned below.

1. Documentation section
2. Link Section
3. Definition Section
4. Global declaration section
5. Function prototype declaration section
6. Main function
7. User defined function definition section

Documentations

Pre process or statements

Global declarations

Main ()

{

Local declarations

Program statements

Calling user defined functions (option to user)

}

User defined functions

Function 1

Function 2

Function n

} Body of the
Main () function

} (Option to user)

Examp

e

```
// Name of Program
```

Documentation section

```
#include<stdio.h>
```

```
#include<conio.h>
```

Preprocessor Directives

```
#define max 100
```

Definition section

```
void add() ;
```

```
int x=100;
```

Global declaration section

```
int main()
```

main () Function section / Entry Point

```
{ int a=100;
```

Variable declaration

```
printf("Hello Main");
```

```
return 0;
```

```
}
```

Body of Main function

```
void add(){
```

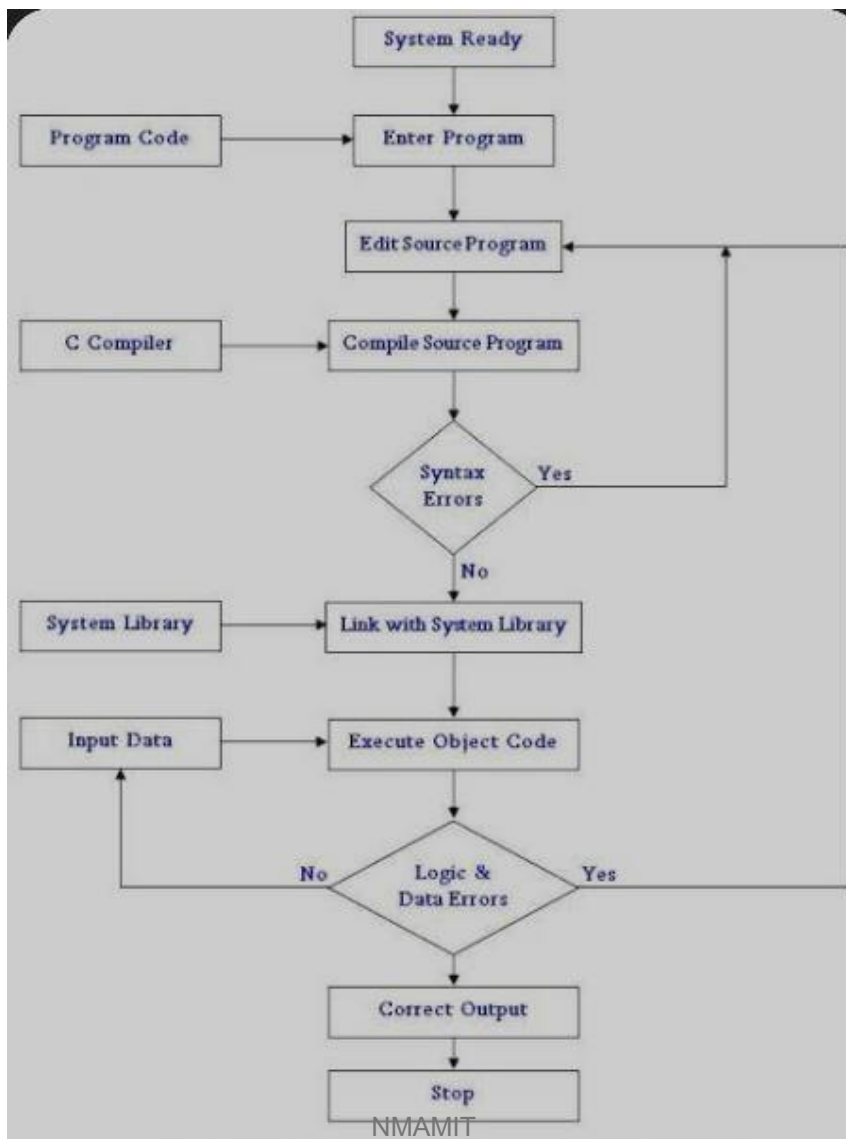
```
printf("Hello add");
```

```
}
```

Function Definition

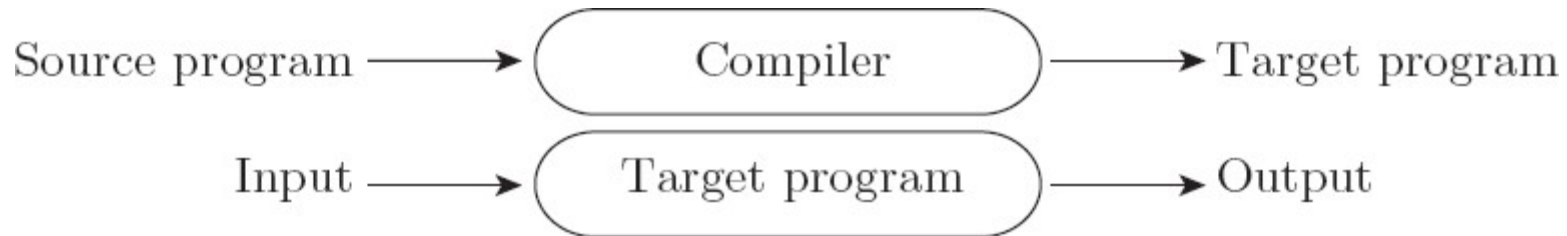
Compilation Process:

- Starting with source code (e.g. C) and converting it into machine code that the computer can run.
- When using our IDE, the process appears like this:



Compilation:

- The compiler translates the source program into an equivalent target program (machine code), then stops running.
- Often, a high-level language (source code) is translated into machine language (object code).
- The operating system later executes the target program on the machine.
- Compiled programs generally have better performance than interpreted programs.



Character set

- The set of valid characters recognized by the C compiler to form programs.
- Used to create keywords, identifiers, constants, strings, and operators.

Categories:

- **Alphabets:**
 - Uppercase letters: A–Z
 - Lowercase letters: a–z
- **Digits :** 0-9
- **Special Characters:** Examples :– + - * / = % ; , { } () [] #
- **White Space Characters:** Space, tab (\t), newline (\n), etc..

C Tokens

- ❖ The smallest individual units in a C program, recognized by the compiler.
- ❖ Tokens are the building blocks of C programs.

Categories of Tokens:

- ❖ **Identifiers**- Names given to variables, functions, arrays, etc..
Must start with a letter or underscore; cannot be a keyword.
- ❖ **Keywords** – Predefined reserved words with special meaning.
 - Examples: int, return, if, while, float
- ❖ **Constants** – Fixed values that do not change during execution.
 - Types: Integer constants, floating-point constants, character constants, string constants.

C Tokens

- ❖ **Strings** – Sequence of characters enclosed in double quotes.
Example: "Hello World"
- ❖ **Operators** – Symbols that perform operations.
Types: Arithmetic (+, -), Relational (<, >), Logical (&&, ||), etc..
- ❖ **Punctuators / Separators** – Symbols that separate statements or elements.
Examples: ;, ,, {}, (), []

C TOKENS EXAMPLE PROGRAM:

```
int main()
{
    int x, y, total;
    x = 10, y = 20;
    total = x + y;
    printf ("Total = %d \n", total);
}
```

where,

- ✓ main – identifier
- ✓ {, }, (,) – delimiter
- ✓ int – keyword
- ✓ x, y, total – identifier
- ✓ main, {, }, (,), int, x, y, total – tokens
- ✓ 10, 20 are the Integer Constants

KEYWORDS IN C LANGUAGE:

- Keywords are predefined words in a C compiler.
- Each keyword is meant to perform a specific function in a C program.
- Since keywords are referred names for compiler, they can't be used as variable name. C language supports 32 keywords. Examples are

<u>auto</u>	<u>double</u>
<u>int</u>	<u>struct</u>
<u>const</u>	<u>float</u>
<u>short</u>	<u>unsigned</u>
<u>break</u>	<u>else</u>
<u>long</u>	<u>switch</u>
<u>continue</u>	<u>for</u>
<u>signed</u>	<u>void</u>

<u>case</u>	<u>enum</u>
<u>register</u>	<u>typedef</u>
<u>default</u>	<u>goto</u>
<u>sizeof</u>	<u>volatile</u>
<u>char</u>	<u>extern</u>
<u>return</u>	<u>union</u>
<u>do</u>	<u>if</u>
<u>static</u>	<u>while</u>

IDENTIFIERS IN C LANGUAGE:

Identifiers are the names provided to the elements of a program such as variables, functions and arrays are examples for identifiers.

e.g. x is a name given to integer variable in above program.

RULES FOR CONSTRUCTING IDENTIFIER NAME IN C:

- First character should be an alphabet or underscore.
- Succeeding characters might be digits or letter.
- Punctuation, spaces and special characters aren't allowed except underscore.
- Identifiers should not be keywords.
- Identifiers are case sensitive.
- Ex: valid identifiers: salary, num, _sum, int_num, avg_salary, num6
- Invalid : int, 9num, n@um

C -Constants

- C Constants are also like normal variables. But, only difference is, their values can not be modified by the program once they are defined.
- Constant is a fixed value assigned to a variable which cannot be modified in a program. They are also called as literals.
- Constants may be belonging to any of the data type.

Syntax: **const data_type variable_name;**

Ex: **const int a=3.14; const char letter= 'n' ;**

TYPES OF C CONSTANT:

- ❖ **Integer constants** : These constant must have digits without decimal point and it be either negative or positive.
- ❖ **Real or floating-point constants** : These constant must have digits along with the decimal point and it be either negative or positive.
- ❖ **Octal & Hexadecimal constants**: These are the numerical constants which start with 0 and 0x respectively.
- ❖ **Character constants** : These constants are a single alphabet, single digit or single special symbol enclosed within single quotes.
- ❖ **String constants** : These constants are a single or several alphabet(s), single or several digit(s) or single or several special symbols or combination of these enclosed within double quotes.
- ❖ **Backslash character constants** : C provides backslash constants which perform specific actions. Ex: '\n' newline, '\t' tab space, '\0' null character etc..

Constant type	data type (Example)
Integer constants	int (53, 762, -478 etc.) unsigned int (5000u, 1000U etc.) long int, long long int (483,647 2,147,483,680)
Real or Floating point constants	float (10.456789) double (600.123456789)
Octal constant	int (Example: 013 /*starts with 0 */)
Hexadecimal constant	int (Example: 0x90 /*starts with 0x*/)
character constants	char (Example: 'A', 'B', 'C')
string constants	char (Example: "ABCD", "Hai")

Operators:

- ❖ Operator is a symbol which helps to perform mathematical and logical computations.
- ❖ Operator acts upon operands
- ❖ Major types of operators are as follows:
- ❖ Unary operator: consists on single operator pre-appended with an operand.
Ex: +a
- ❖ Binary operator: consists of two operands joined by binary operator. Ex: a+b
- ❖ Ternary operator: consists of 3 operands joined by conditional expression operator. Ex: (a>b)?a:b

DATA TYPES IN C

- ❖ A **data type** in programming is a classification that specifies which type of value a variable can hold and how the data is stored and manipulated within a program.
- ❖ Data types define the operations that can be performed on the data, the meaning of the data, and the way values of that type can be stored.
- ❖ In C, data types are essential as they determine how much memory is allocated for a variable and what kind of operations can be performed on that variable.
- ❖ C' supports four basic data types:

Data type	Keyword	Size (in bytes)
Integer	int	2
Real (Floating point)	float	4
Double precision real	double	8
Character	char	1

Data type modifiers:

The storage size in bytes and range of values being represented by basic data types can be modified with the help of the following **modifiers** or **qualifiers** as prefixes.

- 1.signed
- 2.unsigned
- 3.long
- 4.short

Modifier	Size (Bytes)	Range of values
int	2	-32768 to +32767
signed int	2	-32768 to +32767
unsigned int	2	0 to 65535
short int	2	-32768 to +32767
long int	4	-2147483648 to +2147483647
unsigned short int	2	0 to 65535
unsigned long int	4	0 to 4294967295
char	1	-128 to +127
signed char	1	-128 to +127
unsigned char	1	0 to 255
float	4	-3.4E+48 to +3.4E+48
double	8	-1.7E+308 to +1.7E+308
long double	16	-3.4E-4932 to +1.1E+4932

What is a Variable?

- A variable is an identifier which is used to store some value. Variables are the named storage location that holds a data.
- Constants can never change at the time of execution.
- Variables can change during the execution of a program and update the value stored inside it.
- When you declare a constant or a variable, the compiler reserves a memory location in which to store the value of the constant or variable.

RULES FOR NAMING C VARIABLE:

- First character must be letter or underscore, and then it can have considerable number of digits, letter, underscore.
- Keywords cannot be used as identifiers.
- Variables are case sensitive, No special symbols are allowed other than underscore.
- sum, height, _value are some examples for variable name

Following are the examples of valid variable names in a 'C' program:

- height or HEIGHT
- _height
- _height1
- My_name

invalid variable names in a 'C' program:

- 1height
- Hei\$ght
- my name

DECLARING & INITIALIZING C VARIABLE:

- Variables should be declared in the C program before to use.
- Memory space is not allocated for a variable while declaration. It happens only on variable definition.
- Variable initialization means assigning a value to the variable.

Type	Syntax
Variable declaration	<code>data_type variable_name;</code> Example: <code>int x, y, z; char flat, ch;</code>
Variable initialization	<code>data_type variable_name = value;</code> Example: <code>int x = 50, y = 30; char flag = 'x', ch='l';</code>

Symbolic Constants:

- ‘C’ allows defining a variable name as having a constant value using a preprocessor directive `#define`
- Such pre-processor statements are placed at the beginning of the program and are not a part of the C program.
- Such statements begin with the `#` **symbol** and hence do not end with a semicolon.
- A symbolic constant can be used to define a numeric constant or a character/string constant.
- Once defined, a symbolic constant’s value can be used at many places in the program.

The syntax of symbolic constant is:

<code>#define</code>	<code>symbolic_name</code>	value of constant
----------------------	----------------------------	-------------------

```
#define PI 3.141
```

```
#define CLASS “H Section”
```

Input-output Operations

- Every computer program has to take some data as **input** and print/display the processed data as **output**.
- **There are two methods of data input:**
 1. **Non-Interactive method:** Values are assigned to the variables in the program itself. Example : `a=10;`
 2. **Interactive Method:** Data is supplied to the computer by the user through standard input device like the key board.

Examples: the input functions
- To use these functions we attach the header file to our C program by using the preprocessor directive **#include <stdio.h>** at the beginning of the program.
- **#include <stdio.h>** tells the compiler to search for the file named **stdio.h** and place its contents at this point in the program. The contents of the header file become a part of the source code when it is compiled.

- Some of the standard I/O functions frequently used in ‘C’ are:

scanf()	printf()	putchar()
getchar()	getch()	getche()
gets()	puts()	

- There are two types of I/O functions in ‘C’:
 - ❖ Formatted functions
 - ❖ Unformatted functions

	Input function/s	Output function/s
Formatted	scanf()	printf()
Unformatted	getchar(), getch(), gets(), getche() etc.	putchar(), puts() etc.

Formatted I/O function:

- Formatted I/O functions are those which specifies data types and way in which it should be read or printed. `scanf()` and `printf()` are the formatted I/O functions.
- **1) scanf() function:** formatted input function which reads input from the standard input device.
- Syntax :

`scanf("control string", address_list);`

or

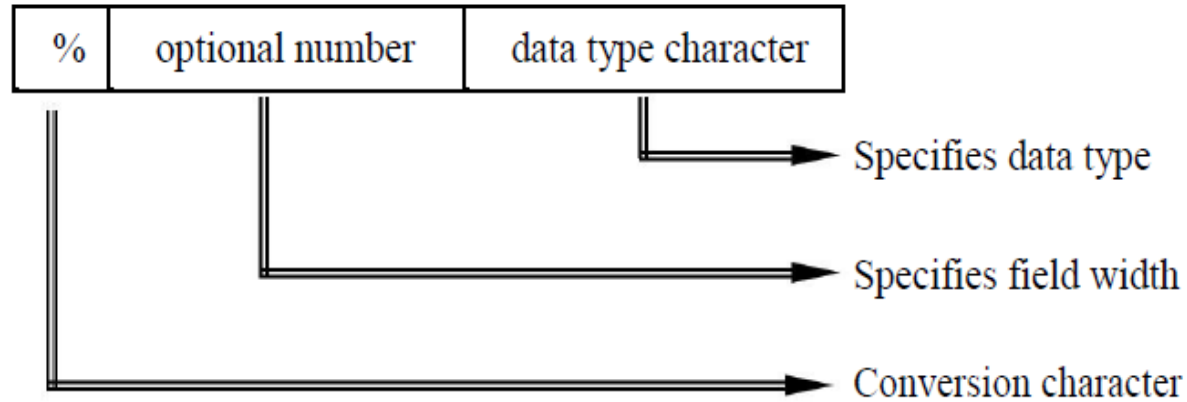
`scanf("control string", &var1, &var2,...            &varn);`

- **"control string"** determines type and form of value to be read.
- **address_list or &var1, &var2,...&varn** - specify the address locations of the variables where the data is stored.

- The table below shows a list of the character groups (or field specifiers) commonly used:

Character group	Character group
<code>%c</code>	Read a single character
<code>%d</code>	Read a decimal integer
<code>%e</code>	Read a floating point number
<code>%f</code>	Read a floating point number
<code>%g</code>	Read a floating point number
<code>%h</code>	Read a short integer
<code>%i</code>	Read a decimal or hexadecimal or octal integer
<code>%o</code>	Read an octal number
<code>%p</code>	Read a pointer
<code>%s</code>	Read a string
<code>%u</code>	Read an unsigned integer
<code>%X</code>	Read a hexadecimal number

- Each character group (field specifier) starts with a % symbol.



- **Examples of scanf() functions:**

- **scanf("%d", &n);** //reads integer value and stores in address of n
- **scanf("%c", &n);** //reads character and stores in address of n
- **scanf("%d%f%c",&num,&avg,&c1);**
- **scanf("%d%f%s",&a,&total,msg);**

Rules that govern the syntax of scanf():

1. Control string must be enclosed with double quotes.
2. For every input variable there must be one character group.
3. Multiple number of character groups are allowed in a control string. They must be separated by blank spaces.
4. Each input variable must in the address_list must be preceded by an ampersand (&) symbol.
5. White spaces may be included in the address_list, but all variables in the address list must be separated by commas.
6. Address_list must not be enclosed within double quotes.
7. The number of character groups, data type and sequence in the control string should match those of the variables in the address_list.
8. A comma should separate the control string and the address_list.

1. Reading Integer numbers:

```
scanf("%d", &n1);
```

```
scanf("%2d", &n1);
```

2. Reading Floating numbers:

```
scanf("%f", &n1);
```

3. Reading single character

```
scanf("%c", &n1);
```

2)printf() function

- Syntax:

printf(“control string”, var_list);

or

printf(“control string”, arg1,arg2,.....argn);

- It refers to display of the processed values or results and messages on the display screen according to the format (way of appearance) as desired by the programmer.
- It is highly desirable that the messages and values must be displayed in the most legible (understandable) form.
- The **printf()** function is a standard formatted output function defined under the header file **stdio.h**.

- Control string consists of three types of items:
 - Characters that will be placed on the screen as they appear.
 - Character groups (format specifiers) that define the output format for display of each item.
 - Escape sequence (backslash constants) such as \n, \t, \b, etc..
- **arg1, arg2,.....argn** are the list of arguments (variables) whose values are formatted and printed according to the specifications of the control string.

- The general format of the character group (format specifier) used in **printf()** is

%w.p type_specifier

where,

- **w** is an integer number that specifies the total number of columns of output value
- **p** is another integer which specifies the number of digits to the right of the decimal point of the floating point number.
- Both **w** and **p** are optional.

Output of integer numbers:

- The general format of the integer group (format specifier) used in **printf()** is

%w type_specifier

- Examples :

<u>printf</u> ("%d",9876)	9	8	7	6		
<u>printf</u> ("%6d",9876)			9	8	7	6
<u>printf</u> ("%2d",9876)	9	8	7	6		
<u>printf</u> ("%-6d",9876)	9	8	7	6		
<u>printf</u> ("%06d",9876)	0	0	9	8	7	6

- The number is written right justified in the given field width.
- Leading blanks will appear as necessary.
- Preceding **w** with a minus sign (-) will cause the number to be displayed in left justification.

Output of floating point numbers:

- The format specification for a **floating-point number** in decimal notation is
%w.pf
- The format specification for a floating-point number in exponential notation is
%w.pe

where,

- w is an integer which specifies the minimum number of positions that are to be used for the display of the value.
- p is an integer which specifies the precision i.e., the number of digits to be displayed after the decimal point.
- w and p are both optional specifications.

Example: Consider the display of the number $y = 98.7654$ under different format

specifiers.

<u>printf</u> ("%7.4f",y);	9	8	.	7	6	5	4
<u>printf</u> ("%7.2f",y);			9	8	.	7	7
<u>printf</u> ("%-7.2f",y);	9	8	.	7	7		
<u>printf</u> ("%f",y);	9	8	.	7	6	5	4

<u>printf</u> ("%10.2e",y);			9	.	8	8	e	+	0	1		
<u>printf</u> ("%11.4e",-y);	-	9	.	8	7	6	5	e	+	0	1	
<u>printf</u> ("%-10.2e",y);	9	.	8	8	e	+	0	1				
<u>printf</u> ("%e",y);	9	.	8	7	6	5	4	0	e	+	0	1

For both the type of notations,

- ◆ The default precision is 6 decimal places.
- ◆ The negative numbers will be printed with a minus sign.
- ◆ The value when displayed, is rounded off to **p** decimal places and printed right justified in the field of **w** columns.
- ◆ Padding the leading blanks with zeros can be done by preceding **w** with a zero.
- ◆ Printing with left-justification is possible by putting a minus sign before **w**.

- A special field specification character “**%*.*f**” that lets the user define the field size **during run-time** may also be used.
- For example:

printf (“%*.*f”,7,2,num);

is same as

printf (“%7.2f”,num);

- The advantage of this format is that the values for the **width and precision** may be supplied during run-time, thus making the format a dynamic one.

Printing of Single Character:

- Using the formatted output, a single character can be
- displayed in a desired position using the format.

% w c

- The character will be right justified in the field of **w** columns.
- The display can be made left justified by placing a minus sign before **w**.
- The default value of **w** is 1.

Unformatted I/O functions

- Unformatted I/O functions are those which do not specify data types and way in which it should be read or printed.
- Input: `getchar()`, `gets()`, `getch()`, `getche()`
- Output: `putchar()`, `puts()`

getchar()

- Function which reads single character at a time during run time.
- when executed waits until any character of the keyboard is keyed in followed by pressing the ENTER (\n) key. Only after ENTER key is pressed, execution is returned to the next statement of program.
- The **getchar** function echoes (displays on monitor) the character that has been keyed in.

Example: char ch;

syntax:

ch = getchar();

- Suppose we enter the character 'y' through the keyboard, 'y' is assigned to ch.

putchar()

- function which displays single character at a time during run time.
- syntax: putchar(ch);
- Where ch is of type char variable.
- Ex: it reads a single character and stores in a variable named as character and displays it.

```
#include <stdio.h>

int main() {
    char ch;
    ch=getchar();
    putchar(ch);
    return 0;
}
```

gets()

- It is another function which is used to **read a string**.
- It accepts all the characters on the keyboard as input and will terminate only when the ENTER key (\n) is pressed.
- **Syntax: gets(strting);**

puts()

- Displays string at a time.
- This statement displays the string on to the display terminal and appends a new line character(\n) in the end.
- **Syntax: puts(string);**
- Where string is string variable name
- Ex: it reads a string and store in a variable named as string and displays it.

```
#include <stdio.h>

int main() {
    char string[10];
    gets(string);
    puts(string);
    return 0;
}
```

OUTPUT:

Hello

Hello

Other functions that are used to read single character

- Other functions that are used to read single variables are (available in **conio.h**):
- **getch()**: Enables the user to enter only one character through the keyboard without pressing the enter key.
- **getche()**: Enables the user to enter only one character through the keyboard without pressing the enter key, and will **echo** (display on monitor) the character typed in. (The last letter 'e' in **getche()** stands for echo)

Other functions that are used to read group of character

- Suppose we want to read the string “Handsome” and assign it to a string variable name

char name[20];

scanf(“%10c”,name);

OR

scanf(“%s”,name);

- Instead of the character group %10c if %6c is used, then only first 6 characters of Handsome i.e., Handso will be assigned to name.
- Suppose for the above first scanf statement we enter Balaguru Swamy, only first 10 characters will be stored in name i.e., Balaguru S Whereas with the second scanf statement, only Balaguru will be stored in name. (i.e., when %s is used, if white space is encountered, scanf will terminate)

Reading Mixed Data Types(Mixed mode Input):

- The scanf() function can be used to read more than one type of data (mixed mode).
- In such situations, the programmer should take care of the
 - (1) number of input variables
 - (2) Data type
 - (3) Order.
- If there is a type mismatch, scanf() function does not read the values input.
- **Example:** Consider the scanf() statement

```
scanf(“%d %c %f %s”,&count,&code,&ratio,name);
```
- The input values entered must be of the form 15 p 1.575 coffee for successful reading of the values respectively to count, code, ratio and name.
- For the above example, on successful reading, the scanf() function will return a value 4 (i.e., the number of data types successfully read).

- Suppose the input values to the above example are of the form:

15 p coffee 1.575

- Then, **scanf()** function will return a value 2 because only two data types have been successfully read.
- There is **match** between the control string parameters and `address_list` parameters only for first two input values.
- The 3rd data was expected to be a floating type number, but a string data type has been encountered. So **scanf()** terminates the scan soon after reading the first two data types.
- If the input values for the above **scanf()** statement were of the form,

20 150.25 motor

- The return value would be 1 since only one data (first data) has been correctly read.

Printing of Strings:

- The format specification for printing strings is similar to that of floating point numbers.

% w.p s

where

- **w** specifies the field width for display.
- **p** specifies that only first p characters of the string are to be displayed.
- The display is right justified.

Consider a string consisting of 16 characters (including blanks)
 “BANGALORE 560010”

|

Specification	Output																			
	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0
%s	B	A	N	G	A	L	O	R	E		5	6	0	0	1	0				
%20s					B	A	N	G	A	L	O	R	E		5	6	0	0	1	0
%20.10s											B	A	N	G	A	L	O	R	E	
%.5s	B	A	N	G	A															
%-20.10s	B	A	N	G	A	L	O	R	E											
%5s	B	A	N	G	A	L	O	R	E		5	6	0	0	1	0				

OPERATORS IN C

- 1. Arithmetic Operators**
- 2. Relational Operators**
- 3. Logical Operators**
- 4. Assignment Operators**
- 5. Increment and Decrement Operators**
- 6. Conditional Operators**
- 7. Bitwise Operators**
- 8. Special Operators**



OPERATORS IN C

(1) Arithmetic Operators

- The basic arithmetic operations addition, subtraction, multiplication and division can be performed on any built-in data type of C using these operators.
- Another operator, the modulus operator is added to the list of Arithmetic operators. It is used to find the remainder after integer division.



OPERATORS IN C

Operator	Meaning
+	Addition
-	Subtraction or Unary Minus
*	Multiplication
/	Division
%	Modulus or Modulo Division



OPERATORS IN C

Note:

1. The modulus operator % can be used only for integer operands and not for floating point operands.
2. C does not have a operator for exponentiation. This can be performed by a library function **pow()**. Examples of arithmetic operators are:

$a - b$

$a + b$

$a * b$

a / b

$a \% b$

$-a * b$



where 'a' and 'b' are operands

OPERATORS IN C

(a) Integer Arithmetic Operator

1. When the both the operands in a single arithmetic expression are integers, the expression is called **integer expression** and the operation is called **integer arithmetic**.
2. Integer arithmetic always yields an integer value.

Examples: if $a = 17$ and $b = 4$, the following are the results of integer arithmetic.

$$a - b = 13$$

$$a + b = 21$$

$$a * b = 68$$

$$a / b = 4 \text{ (decimal part truncated)}$$

$$a \% b = 1 \text{ (remainder of division)}$$



OPERATORS IN C

```
/****Use of Modulus operator****/  
#include <stdio.h>  
#include <conio.h>  
main()  
{  
int months,tot_days,days;  
clrscr();  
printf("\n\n\tEnter the no. of days:");  
scanf("%d",&tot_days);  
months=tot_days/30;  
days=tot_days%30;  
printf("\n\n\t%d tot_days = %d months & %d days", tot_days, months, days);  
getch();  
}
```



OPERATORS IN C

(b) Floating point OR Real Arithmetic:

1. An arithmetic expression involving only real (or floating point) operands is called Real Arithmetic.
2. A real operand may assume values either in the decimal or exponential form.
3. The result of real arithmetic operation is an approximation of the correct result to the number of significant digits permissible.
4. The modulus operator % cannot be used with real operands.



OPERATORS IN C

Examples: if $x = 8.3$, $y = 3.2$ and z are floats, then we will have:

$$z = x + y = 11.500000$$

$$z = x / y = 2.593750$$

$$z = -y/x = -0.385542$$



OPERATORS IN C

(c) Mixed-mode Arithmetic:

1. The arithmetic operation in which one of the operand is real and the other is integer, is called mixed-mode arithmetic operation.
2. The value of such an expression is a float.
3. But if the result is assigned to an int variable, the decimal portion of the result is truncated and only integer portion is assigned to the int variable.

Example: `int n;`

$$n = 5/3 + (3/2.0)$$

$$n = 1 + (1.5) = 2.5$$

But 'n' is an integer variable, so only the integer part of 2.5 i.e., 2 is assigned to 'n'. Therefore, `n = 2`.

OPERATORS IN C

(2) Relational Operator:

- These are used to compare two operands. For example, compare the ages of two persons, compare the prices of two or more items, etc..
- They result in either a TRUE (Non-zero) value or FALSE (Zero) value.



OPERATORS IN C

	Operator	Meaning	Precedence	Associativity
	<	Lesser than	1	L to R
	<=	Less than or equal to	1	L to R
	>	Greater than	1	L to R
	>=	Greater than or equal to	1	L to R
	==	Equal to	2	L to R
	!=	Not equal to	2	L to R



OPERATORS IN C

Examples:

`x > y`

`age == 25`

`cost <= 125.5`

`i != 10`

`days < 365`

Relational Expressions:

1. A simple relational expression contains only one relational operator. Its syntax is as follows:



Where ae1 and ae2 are arithmetic expressions, which may be simple constants, variables or combination of them.



OPERATORS IN C

1. When arithmetic expressions are used on either side of a relational operator, the arithmetic expressions will be evaluated first and then the results compared. That is, arithmetic operators have a higher priority over relational operators.
2. Relational expressions are used in *decision statements* such as **if** and **while** to decide the course of action of a running program.



OPERATORS IN C

Suppose a and b are two float point variables holding values 4.5 and 10.0,

Expression	Interpretation	Value
a <= b	TRUE	1
a < -b	FALSE	0
a*9 > 0	FALSE	0
a + 5 < b * 2	TRUE	1
a*b == b*a	TRUE	1



OPERATORS IN C

(3) Logical Operators

Logical operators are used in C to take decisions.
C has the following three logical operators:

Operator	Meaning	Precedence	Associativity
&&	Logical AND	2	L to R
	Logical OR	3	L to R
!	Logical NOT	1	L to R

&& and || are binary operators and ! is a unary operator.

1. The result of these operators is either TRUE (ONE) or FALSE (ZERO).
2. The logical operators are used to connect one or more relational expressions.
-  3. Such an expression which combines two or more relational expressions is called as **logical expression** or **compound relational expression**.

OPERATORS IN C

While using logical AND

- if both the operands are TRUE, then the result of the logical expression is TRUE.
- even if one of the operands is FALSE, then the result of the logical expression is FALSE.

While using logical OR

- Only if both the operands are FALSE, then the result of the logical expression is FALSE.
- Even if one of the operands is TRUE, then the result of the logical expression is TRUE.

While using logical NOT

- the compliment of the operand is obtained. That is, if the operand is TRUE, then the result will be FALSE or vice-versa.



OPERATORS IN C

The results of the operands are shown in Truth table shown below:

Op1	Op2	Value of Expression			
		Op1 && Op2	Op1 Op2	!Op1	!Op2
T	T	T	T	F	F
T	F	F	T	F	T
F	T	F	T	T	F
F	F	F	F	T	T

Examples of Logical Expressions:

1. `if(age > 55 && salary < 50000)`

 2. `if(number < 0 || number > 100)`

OPERATORS IN C

Suppose a, b and c are integers having values, 2, 4 and 3,

a && b || c && (!b)

2 && 4 || 3 && (!4)

2 && 4 || 3 && 0

1 || 3 && 0

1 || 0

1



OPERATORS IN C

(4) Assignment Operators:

- `a=10` is an assignment statement
- Value of 10 has been assigned to `a`.

Shorthand Assignment Operators:

1. Shorthand assignment operators associated with arithmetic operations have already been seen.
2. The bitwise operators can also be written in shorthand form.



OPERATORS IN C

Operator	Assignment Expression	Shorthand Assignment
+	$a = a + b$	$a += b$
-	$a = a - b$	$a -= b$
*	$a = a * b$	$a *= b$
/	$a = a / b$	$a /= b$
%	$a = a \% b$	$a \% = b$
&	$a = a \& b$	$a \& = b$
	$a = a b$	$a = b$
^	$a = a ^ b$	$a ^ = b$
<<	$a = a << b$	$a << = b$
>>	$a = a >> b$	$a >> = b$



OPERATORS IN C

(5) Increment and Decrement Operators:

1. C has two very useful operators generally not found in other languages.
2. They are: **Increment Operator** denoted by **++** and **Decrement Operator** denoted by **--**
3. The increment operator **++** adds 1 to the operand while **--** subtracts 1 from the operand.
4. These operators act only on integer operands.
5. Both are **unary operators** taking the form

++m or m++

--m or m--



OPERATORS IN C

6. Increment and Decrement Operators are used extensively in **for** and **while** loops.
7. `++m` and `m++` (or `--m` and `m--`) mean the same when they are used independently in statements which are not expressions.



OPERATORS IN C

They behave differently when they are used in expressions on right-hand side of an assignment statement as shown in table below

Expression	Meaning	Known as
<code>x = ++a;</code>	<code>a = a+1;</code> <code>x = a;</code>	Pre increment – Integer variable on RHS will be incremented first, then it's new value is assigned to variable on LHS.
<code>x = a++;</code>	<code>x = a;</code> <code>a = a+1;</code>	Post increment – Present value of integer variable on RHS is first assigned to variable on LHS, then integer on RHS is incremented.
<code>y = --b;</code>	<code>b = b-1;</code> <code>y = b;</code>	Pre decrement – Integer variable on RHS will be decremented first, then it's new value is assigned to variable on LHS.
<code>y = b--;</code>	<code>y = b;</code> <code>b = b-1;</code>	Post decrement – Present value of integer variable on RHS is first assigned to variable on LHS, then integer on RHS is decremented.



OPERATORS IN C

```
/** Use of increment and decrement operators */  
#include <stdio.h>  
int main()  
{  
    int a=10,b=12;  
    printf("Given a = %d and b = %d",a,b);  
    printf("\na++ = %d",a++);  
    printf("\nb-- = %d",b--);  
    printf("\nNow a = %d",a);  
    printf("\nNow b = %d",b);  
    printf("\n++a = %d",++a);  
    printf("\n--b = %d",--b);  
    return 0;  
}
```



OPERATORS IN C

```
/** Use of increment and decrement operators */  
#include <stdio.h>  
int main()  
{  
    int a=10,b=12;  
    printf("Given a = %d and b = %d",a,b);  
    printf("\na++ = %d",a++);  
    printf("\nb-- = %d",b--);  
    printf("\nNow a = %d",a);  
    printf("\nNow b = %d",b);  
    printf("\n++a = %d",++a);  
    printf("\n--b = %d",--b);  
    return 0;  
}
```



Given a = 10 and b = 12

a++ = 10

b-- = 12

Now a = 11

Now b = 11

++a = 12

--b = 10

OPERATORS IN C

(6) Conditional Operator: (Ternary)

- It takes three operands.
- There is only one such operator in 'C' and it is called the '**Conditional Operator**'.

It uses character pair	? :
Its general syntax is	test_exp ? exp1 : exp2

where, **test_exp** is a test condition, usually a relational expression like $a > b$ **exp1** and **exp2** can be any valid arithmetic expressions or variables or constants.



OPERATORS IN C

The `? :` operator pair works as follows:

- If the result of the `test_exp` is `TRUE`, `exp1` is evaluated and the value of `exp1` is the value of conditional operation.
- If the value of `test_exp` is `FALSE`, `exp2` is evaluated and the value of `exp2` is the value of the conditional operation.

Example

```
m = x > y ? a : b - 146;
```

- if `x > y` is `TRUE`, the value of `'a'` is assigned to `'m'`,
- or if `x > y` is `FALSE`, the value of `'b-146'` is assigned to `'m'`.



OPERATORS IN C

(7) Bitwise Operators

1. All the data stored in the computer memory are in sequences of bits (0s and 1s).
2. Some applications require the manipulation of these bits.
3. Manipulation of individual bits is carried out in machine language or assembly language.
4. 'C' provides **six operators** to perform **bitwise operations**.
5. These operators work only with **int** and **char** data-types. They cannot be used with floating point numbers.



OPERATORS IN C

The six bitwise operators in 'C' are:

Operator	Meaning
&	Bitwise AND
	Bitwise OR
^	Exclusive OR (XOR)
~	1's complement
<<	Left shifting of bits
>>	Right shifting of bits



OPERATORS IN C

(a) Bitwise AND:

Result of bitwise AND is 1 when both the bits are 1, otherwise it is zero.

Considering BCD (Binary Coded Decimal).

int m = 120, n = 060, p;		
Equivalent BCD of m = 120 is		001 010 000
Equivalent BCD of n = 060 is		000 110 000
p = m & n	is	000 010 000



OPERATORS IN C

(b) Bitwise OR (|):

Result of bitwise OR is 1 when one of the bits is 1, otherwise (when both bits are 0s) it is zero.

Examples:		
Considering normal binary digits.		
int a = 6, b = 4;		
Equivalent binary value of a = 6 is		0000 0110
Equivalent binary value of b = 4 is		0000 0100
a b	is	0000 0110



OPERATORS IN C

(c) Exclusive OR (XOR ^):

Result of bitwise XOR is 1 if the bits are different (1 and 0 or 0 and 1), otherwise (when both bits are 0s or both bits are 1s) it is zero.

b1	b2	b1 ^ b2
0	0	0
0	1	1
1	0	1
1	1	0

Examples

Considering normal binary digits.

int a = 6, b = 4;

Equivalent binary value of a = 6 is

0000 0110

Equivalent binary value of b = 4 is

0000 0100

a ^ b

is

0000 0010



OPERATORS IN C

(d) One's Complement (~):

- The bitwise complement operator is an unary operator which reverses the state of each bit within an integer or character.
- Each 'zero' get changed to 'one' and each 'one' gets changed to 'zero'.

Example:

`int b = 12;`

1100 (in binary form)

`a = ~b`

0011

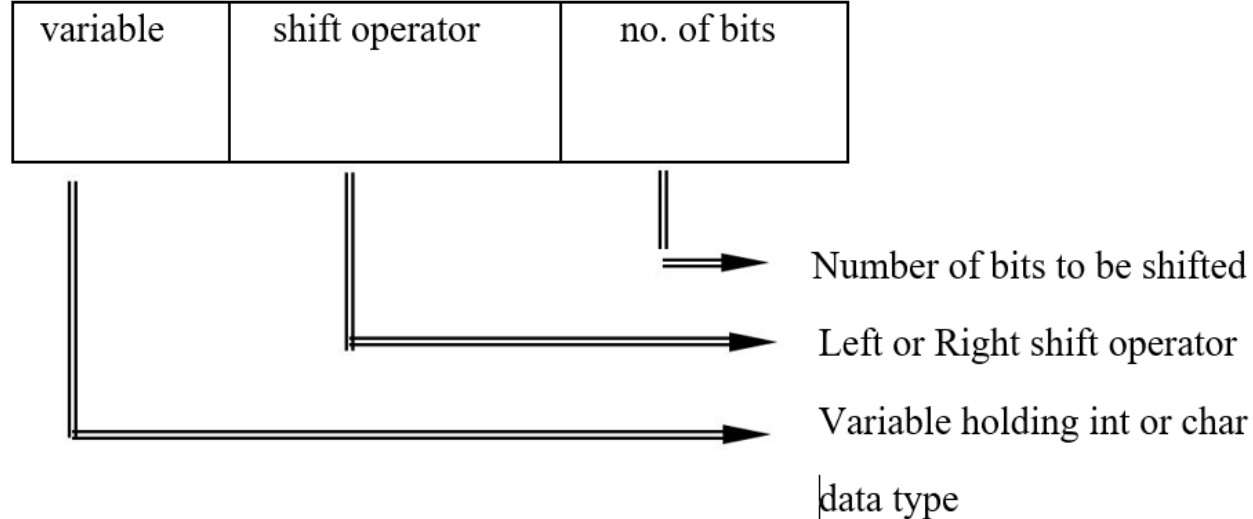


OPERATORS IN C

(e) Left Shift Operator (<<) and Right Shift Operator (>>):

The left shift operator << shifts the bits to the left and the right shift operator >> shifts the bits to the right.

General syntax for bitwise shift operator:



OPERATORS IN C

Example for bitwise left shifting:

Consider an integer variable a = 208

In binary form it is a =

1	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---

a << 1 =

1	0	1	0	0	0	0	0
---	---	---	---	---	---	---	---

Example for bitwise right shifting:

Consider an integer variable a = 208

In binary form it is a =

1	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---

a >> 2 =

0	0	1	1	0	1	0	0
---	---	---	---	---	---	---	---



OPERATORS IN C

(8) Special Operators:

(a) The sizeof() Operator:

- The **sizeof()** operator returns the size (number of bytes) of the operand.
- The operand may be a constant, variable or any valid data-type.
- The operand is written within the parentheses of the sizeof operator.
- It is commonly used to determine the lengths of arrays and structures when their sizes are not known to the programmer.
- It is also used to allocate memory space dynamically to variables during the execution of the program.

Examples: `a = sizeof (int);`
 `y = sizeof (avg);`



OPERATORS IN C

(b) The Comma Operator:

- The comma operator can be used to link the related expressions together.
- A comma linked list of expressions are evaluated left to right and the value of the right most expression is the value of the combined expression.

Example:

value = (x = 10, y = 5, x+y);

first assigns the value 10 to **x**, then assigns 5 to **y**, and finally assigns 15 (i.e., 10+5) to **value**.



HIERARCHY OF OPERATORS

Operator			Description	Associativity	Precedence
()			Function call	L to R	1
[]			Array references		
+			Unary plus		
-			Unary minus		
++			Increment	R to L	2
--			Decrement		
!			Logical negation		
~			Ones complement		
*			Pointer reference		
&			Address		
sizeof()			Size of an object		
(type)			Type cast (conversion)		



HIERARCHY OF OPERATORS

*			Multiplication		
/			Division	L to R	3
%			Modulus		
+			Addition		
				L to R	4
-			Subtraction		
<<			Bitwise left shift		
				L to R	5
>>			Bitwise right shift		
<			Less than		
<=			Less than or equal to		
				L to R	6
>			Greater than		
>=			Grater than or equal to		
==			Equal to		
				L to R	7
!=			Not equal to		



HIERARCHY OF OPERATORS

&			Bitwise AND	L to R	8
^			Bitwise XOR	L to R	9
			Bitwise OR	L to R	10
&&			Logical AND	L to R	11
			Logical OR	L to R	12
? :			Conditional expression	R to L	13
=					
*=	/=	%=			
+=	-=	&=	Assignment operators	R to L	14
^=	=				
<<=	>>=				
,			Comma operator	L to R	15



C EXPRESSIONS

- Expressions are basically operators acting on operands.
- There are rules that govern how the expression gets interpreted by the compiler.
- Such expressions follow C's precedence and associativity rules.
- The precedence of operators determine a rank for the operators.

Precedence and Associativity

Operator	Description	Associativity	Precedence
+ - ++ -- ! ~ *	Unary plus Unary minus Increment Decrement Logical negation Ones complement Pointer reference	R to L	1
& sizeof() (type)	Address Size of an object Type cast (conversion)	L to R	
() []	Function call Array references	L to R	2
* / %	Multiplication Division Modulus	L to R	3
+ -	Addition Subtraction	L to R	4
<< >>	Bitwise left shift Bitwise right shift	L to R	5

Precedence and Associativity

Operator	Description	Associativity	Precedence
< <= > >=	Less than Less than or equal to Greater than Grater than or equal to	L to R	6
== !=	Equal to Not equal to	L to R	7
&	Bitwise AND	L to R	8
^	Bitwise XOR	L to R	9
	Bitwise OR	L to R	10
&&	Logical AND	L to R	11
	Logical OR	L to R	12
? :	Conditional expression	L to R	13
= *= /= %= += -= &= ^= = <<= >>=	Assignment operators	R to L	14
,	Comma operator	R to L	15

OPERATOR PRECEDENCE AND ASSOCIATIVITY

- The *operator precedence* and *associativity rules* specifies the order in which operators in an expression are bound to the operands.

Precedence of Operators:

- The order in which different operators are used to evaluate an expression is called precedence of operators or hierarchy of operators.
- The operators are evaluated from highest precedence to least precedence.

OPERATOR PRECEDENCE AND ASSOCIATIVITY

Associativity of Operators:

Whenever two or more operators having the same precedence or priority occurs in the expression, then the direction order chosen (left to right or right to left) to evaluate an expression is called associativity of operators.

They are classified into two categories based on the direction of evaluation chosen as below:

Left to Right Associativity:

- Here the direction order chosen for evaluation is left to right.
- E.g.: Arithmetic operators, Logical Operators and Relational operators follows left to right associativity.

Right to Left Associativity:

- Here the direction order chosen for evaluation is right to left.
- E.g.: Assignment operator, unary operators and conditional

PRECEDENCE OF ARITHMETIC OPERATORS

- An arithmetic expression without parenthesis will be evaluated from *left to right* using the rules of precedence of operators.
- There are two distinct priority levels of arithmetic operators in C:

High priority * / %

Low priority + -

E.g.: Consider the following evaluation statement

$$x = a - b / 3 + c * 2 - 1$$

when $a=9$, $b=12$, and $c=3$ the statement becomes

$$x = 9 - 12 / 3 + 3 * 2 - 1$$

and is evaluated as follows

Step 1: $x = 9 - 4 + 3 * 2 - 1$

Step 2: $x = 9 - 4 + 6 - 1$

Step 3: $x = 5 + 6 - 1$

Step 4: $x = 11 - 1$

Step 5: $x = 10$

E.g. 2:

C = 2 && 5 || !6

C = 2 && 5 || (!6)

C = (2 && 5) || 0

C = 1 || 0

C = 1

E.g. 3:

12 && 5 && 4 || 3 || !0

12 && 5 && 4 || 3 || (!0)

(12 && 5) && 4 || 3 || 1

(1&& 4) || 3 || 1

(1 || 3) || 1

1 || 1

1

Operators	Priority
!	1
&&	2
	3

E.g. 4:

`11 + 2 > 6 && !0 || 11 != 7 && 11 - 2 <= 5`

`11 + 2 > 6 && (!0) || 11 != 7 && 11 - 2 <= 5`

`(11 + 2) > 6 && 1 || 11 != 7 && 11 - 2 <= 5`

`13 > 6 && 1 || 11 != 7 && (11 - 2) <= 5`

`(13 > 6) && 1 || 11 != 7 && 9 <= 5`

`1 && 1 || 11 != 7 && (9 <= 5)`

`1 && 1 || (11 != 7) && 0`

`(1 && 1) || (1 && 0)`

`1 || 0`

1

Opereators	Priorities
!	1
*, /, %	2
+, -	3
<, <=, >, >=	4
!=, ==	5
&	6
^	7
	8
&&	9
	10

Type Conversion In C

- The process of converting one type of data type to another, say change int to float or float to int is known as data type conversion or type casting.

The general form of type casting is

(data type)variable

Example

```
int a = 3;  
float sum;  
sum = 20/a;
```

Value of 'sum' is 6.00000

```
int a = 3;  
float sum;  
sum = 20/(float)a;
```

Value of 'sum' is 6.666666

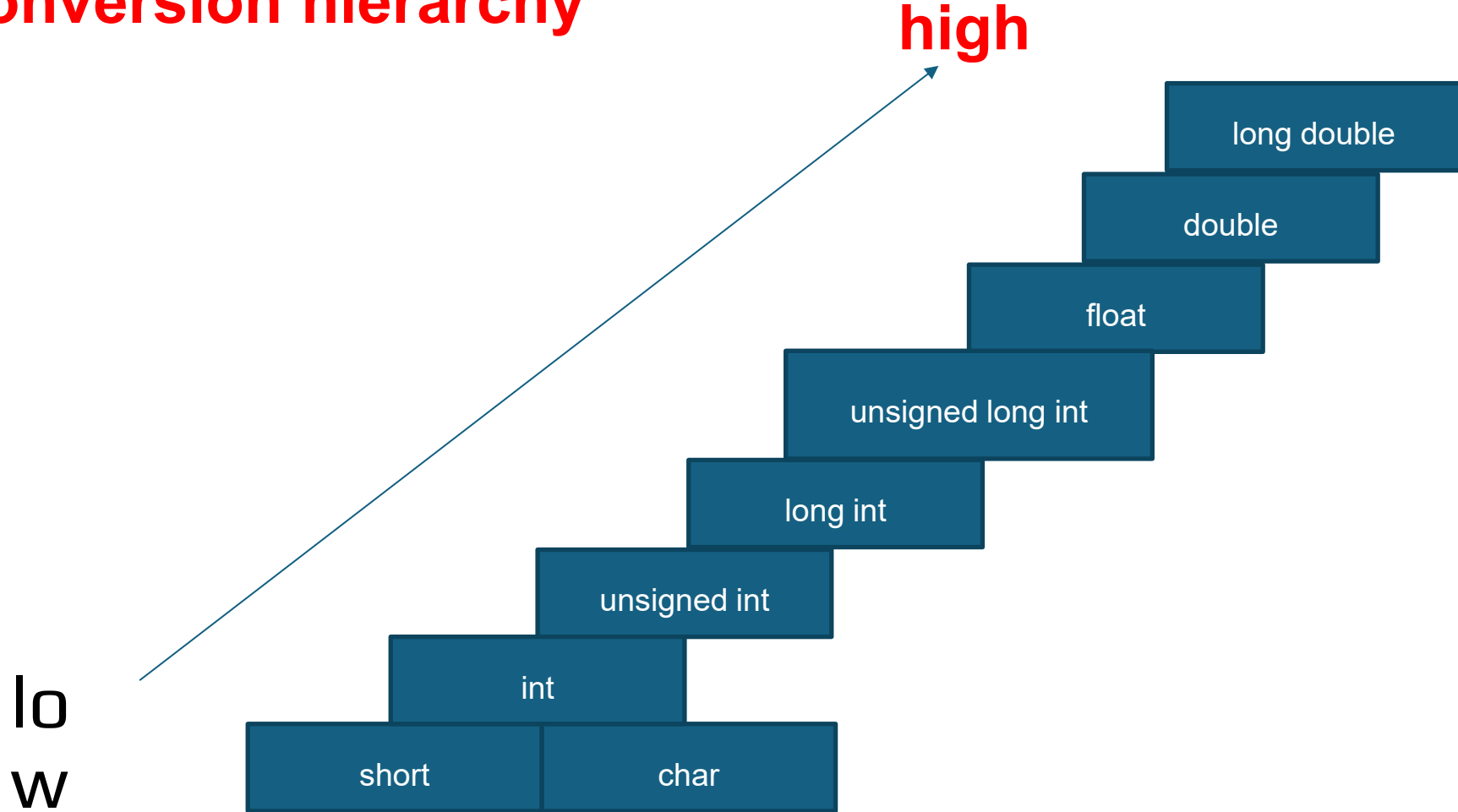
Data type conversion can be done in two ways:

1. Implicit type conversion
2. Explicit type conversion

Implicit type conversion

- This type of conversion is usually performed by the compiler when necessary without any commands by the user.
- Thus it is also called "*Automatic Type Conversion*".
- The compiler usually performs this type of conversion when a particular expression contains more than one data type.
- The compiler first performs integer promotion; if the operands still have different types, then they are converted to the type that appears highest in the following hierarchy.

Conversion hierarchy



Example:

```
#include <stdio.h>
```

```
int main() {  
    int i = 17;  
    char c = 'c'; /* ASCII value is 99 */  
    float sum;  
  
    sum = i + c;  
  
    printf("Value of sum: %f\n", sum);  
  
    return 0;  
}
```

Output:

Value of sum : 116.000000

Explicit type conversion:

In Explicit type conversion, the user explicitly defines within the program the datatype, to which the operands/variables of the expression need to be converted.

```
#include <stdio.h>
int main() {
    double a = 4.5;
    double b = 4.6;
    double c = 4.9;
    int result;
    result = (int)a + (int)b + (int)c; // Explicitly
    casting double to int
    printf("Result = %d\n", result);
    return 0;
}
```

Output: Result = 12

End of Unit 1

